

GEORGE BROWN POLYTECHNIC

Applied A.I. Solutions Development

PREDICTING EMAIL REPLY LIKELIHOOD

An NLP and Deep Learning Approach Using Azure Machine Learning

PROJECT DOCUMENTATION & IMPLEMENTATION EVIDENCE

Supporting evidence showing the main project steps, screenshots, model development, evaluation, and Azure Machine Learning deployment.

PROJECT INFORMATION

Prepared by	Adnan Ahmed
Course	Applied Math for Deep Learning
Professor	Dr. Mahdieh Khalilinezhad
Date	September 2026

EVIDENCE PROFILE

Project Domain	Natural Language Processing (NLP)
Dataset	Enron Email Corpus
Project Sample	20,000 emails
Manual Validation	300 emails
Baseline Model	LSTM
Final Model	DistilBERT
Cloud Platform	Microsoft Azure Machine Learning
Evidence Type	Project Screenshots & Implementation Evidence

PROJECT WORKFLOW

Enron Corpus → Data Preparation → Reply Labeling → Manual Validation
LSTM → DistilBERT → Model Comparison → Azure Deployment

TOOLS & TECHNOLOGIES

Python • TensorFlow • PyTorch • Hugging Face • Azure Machine Learning

Project Overview

This project was developed to predict whether an outbound email is likely to receive a reply. The **Enron Email Corpus** was loaded into **Azure Machine Learning**, where the data was explored, cleaned, sampled, and labeled using rules to identify whether an email received a reply. A manual validation process was also completed to check the quality of the generated labels.

Two deep learning models were developed and evaluated: **LSTM** as the baseline model and **DistilBERT** as the advanced model. Their performance was compared using evaluation metrics, and **DistilBERT** was selected as the final model because it performed better at identifying Reply emails. The final model was saved, registered, and deployed using **Azure Machine Learning**.

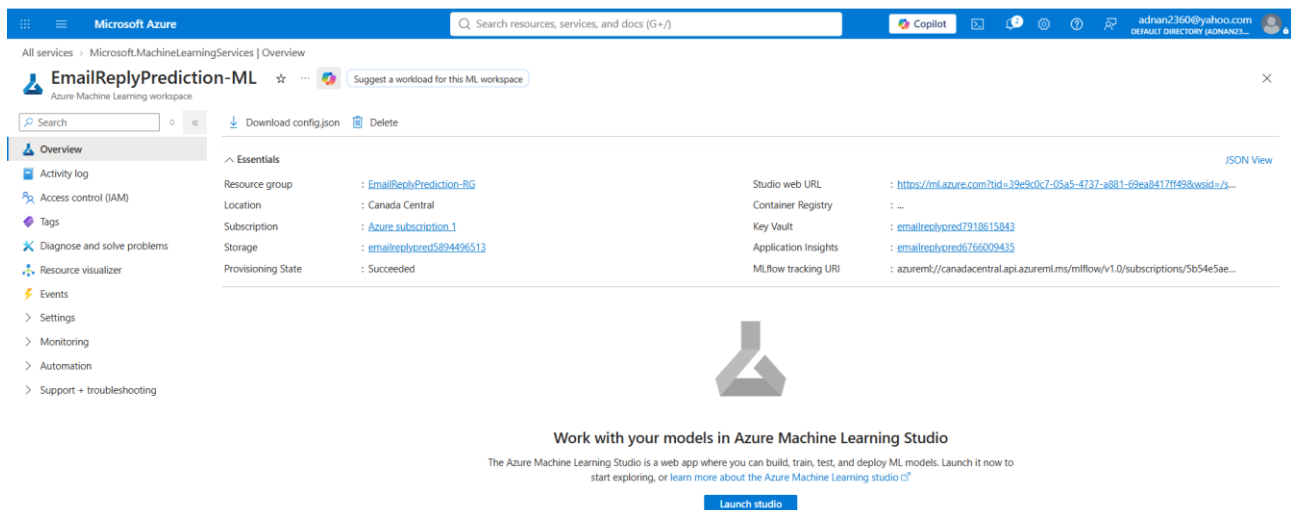
This document presents screenshots and short explanations as evidence of the main steps completed throughout the project.

Step 1 — Set Up Azure Machine Learning

1.1 Create Azure ML Workspace

An Azure Machine Learning workspace was created as the main cloud environment for the project. It will be used to manage the data, notebooks, compute resources, model training, experiments, and trained models throughout the project.

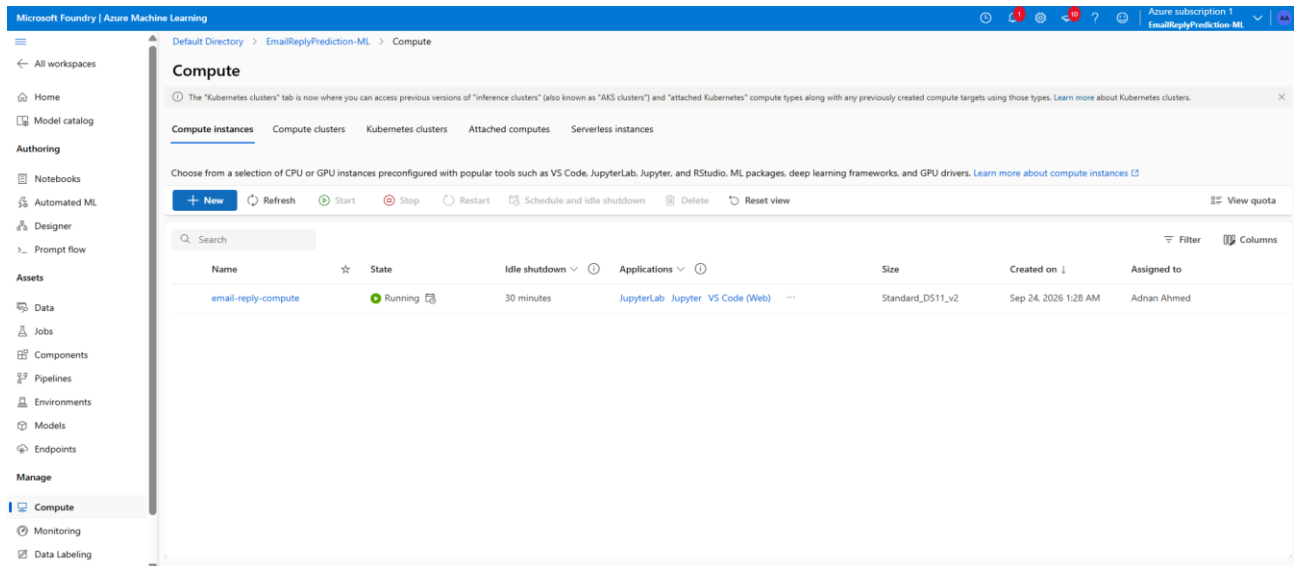
Figure 1. Azure Machine Learning workspace successfully created.



1.2 Create Azure ML Compute Instance

A compute instance was created to provide the cloud computing resources needed to run the project notebooks, process the dataset, and train the machine learning models. Automatic idle shutdown was enabled to help control Azure computing costs.

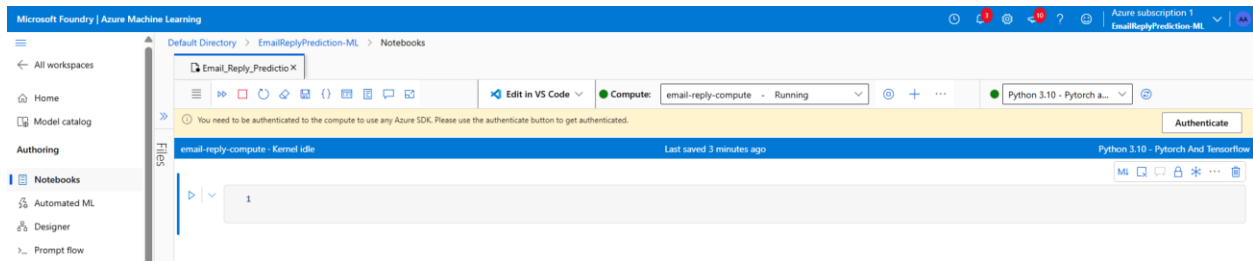
Figure 2. Azure Machine Learning compute instance successfully created and running.



1.3 Create and Connect the Project Notebook

A Jupyter Notebook was created in Azure Machine Learning and connected to the running compute instance. The Python 3.10 PyTorch and TensorFlow environment was selected to support the deep learning models used in this project.

Figure 3. Azure Machine Learning notebook connected to the compute instance and Python environment.

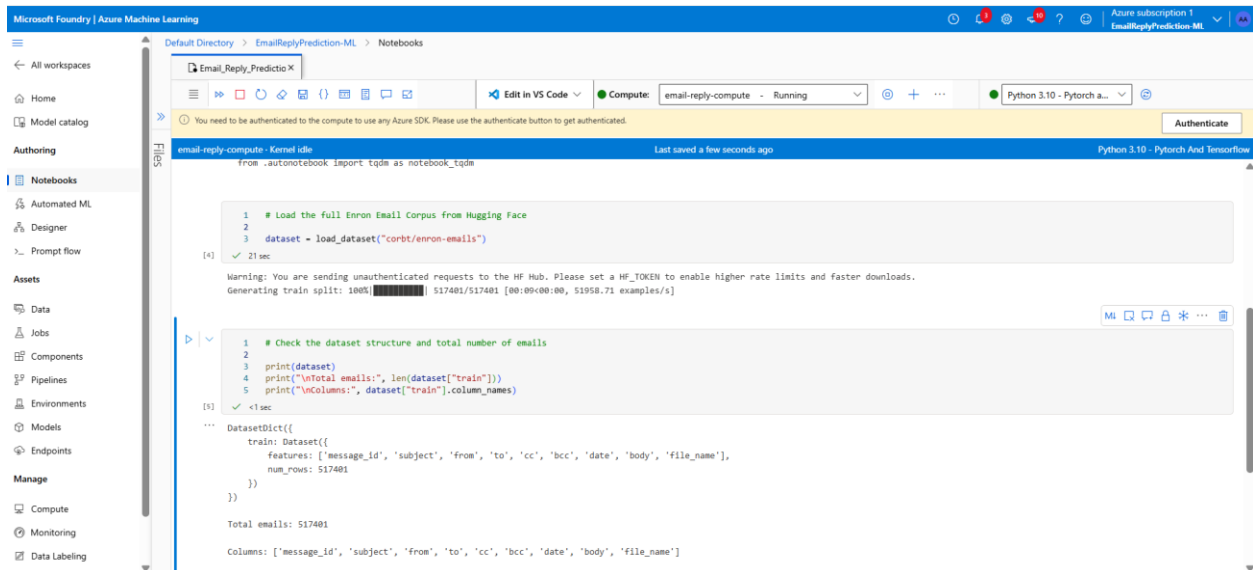


Step 2 — Load the Enron Dataset

2.1 Load and Verify the Original Dataset

The original Enron Email Corpus was loaded directly from Hugging Face into the Azure Machine Learning notebook. The dataset contains **517,401 email records and 9 columns**. Its structure was verified before performing any sampling, cleaning, or preprocessing.

Figure 4. Original Enron Email Corpus successfully loaded and verified.



```
from .autonotebook import tqdm as notebook_tqdm

1 # Load the full Enron Email Corpus from Hugging Face
2
3 dataset = load_dataset("corbt/enron-emails")
4
5 # Check the dataset structure and total number of emails
6 print(dataset)
7 print("\nTotal emails:", len(dataset["train"]))
8 print("\nColumns:", dataset["train"].column_names)

DatasetDict({
  train: Dataset({
    features: ['message_id', 'subject', 'from', 'to', 'cc', 'bcc', 'date', 'body', 'file_name'],
    num_rows: 517481
  })
})

Total emails: 517481

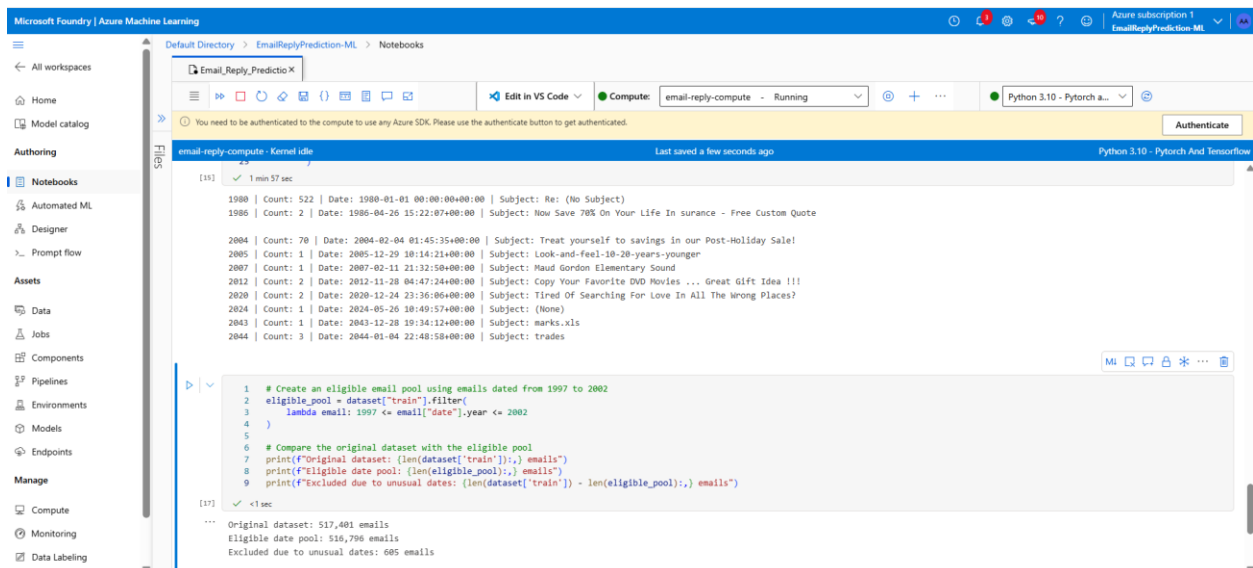
Columns: ['message_id', 'subject', 'from', 'to', 'cc', 'bcc', 'date', 'body', 'file_name']
```

Step 3 — Data Preparation and Sampling

3.1 Create the Eligible Email Pool

Before selecting the project sample, the raw dataset was filtered to create an eligible email pool. Emails dated from **1997 to 2002** were retained, while **605 records with unusual dates** were excluded. The original dataset was kept unchanged for later reference and reply matching.

Figure 5. Eligible email pool created after filtering unusual dates.



```
1980 | Count: 522 | Date: 1980-01-01 00:00:00+00:00 | Subject: Re: (No Subject)
1986 | Count: 2 | Date: 1986-04-26 15:22:07+00:00 | Subject: Now Save 70% On Your Life In surance - Free Custom Quote

2004 | Count: 70 | Date: 2004-02-04 01:45:35+00:00 | Subject: Treat yourself to savings in our Post-Holiday Sale!
2005 | Count: 1 | Date: 2005-12-29 18:14:21+00:00 | Subject: Look-and-feel-18-28-years-younger
2007 | Count: 1 | Date: 2007-02-11 21:12:59+00:00 | Subject: Naul Gordon Elementary Sound
2012 | Count: 2 | Date: 2012-11-28 04:47:24+00:00 | Subject: Copy Your Favorite DVD Movies ... Great Gift Idea !!!
2020 | Count: 2 | Date: 2020-12-24 23:36:06+00:00 | Subject: Tired Of Searching For Love In All The Wrong Places?
2024 | Count: 1 | Date: 2024-05-26 18:49:57+00:00 | Subject: (None)
2043 | Count: 1 | Date: 2043-12-28 19:34:12+00:00 | Subject: marks.xls
2044 | Count: 3 | Date: 2044-01-04 22:48:58+00:00 | Subject: trades

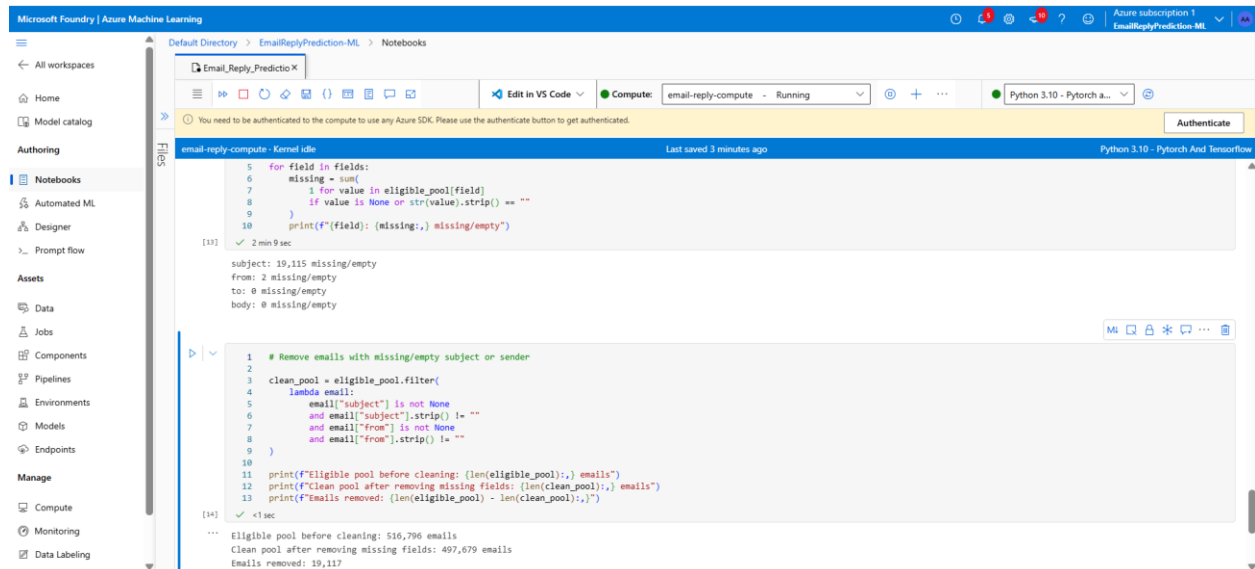
1 # Create an eligible email pool using emails dated from 1997 to 2002
2 eligible_pool = dataset["train"].filter(
3     lambda email: 1997 <= email["date"].year <= 2002
4 )
5
6 # Compare the original dataset with the eligible pool
7 print(f"Original dataset: {len(dataset['train']):,} emails")
8 print(f"Eligible date pool: {len(eligible_pool):,} emails")
9 print(f"Excluded due to unusual dates: {len(dataset['train']) - len(eligible_pool):,} emails")

Original dataset: 517,481 emails
Eligible date pool: 516,796 emails
Excluded due to unusual dates: 605 emails
```

3.2 Remove Emails with Missing Required Fields

Emails with missing or empty subject or sender information were removed because these fields are needed for reply matching. After this cleaning step, **497,679 emails** remained in the eligible pool.

Figure 6. Removal of emails with missing subject or sender information.

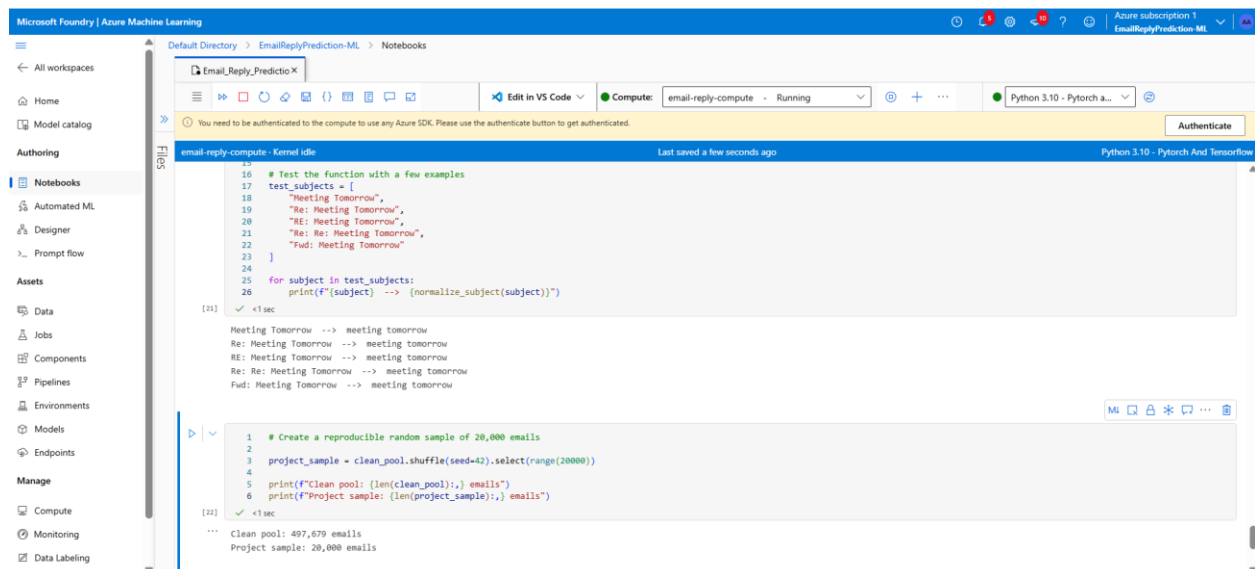


```
1 # Remove emails with missing/empty subject or sender
2
3 clean_pool = eligible_pool.filter(
4     lambda email:
5         email['subject'] is not None
6         and email['subject'].strip() != ''
7         and email['from'] is not None
8         and email['from'].strip() != ''
9 )
10
11 print(f'Eligible pool before cleaning: {len(eligible_pool):,} emails')
12 print(f'Clean pool after removing missing fields: {len(clean_pool):,} emails')
13 print(f'Emails removed: {len(eligible_pool) - len(clean_pool):,}')
14
15 Eligible pool before cleaning: 516,796 emails
16 Clean pool after removing missing fields: 497,679 emails
17 Emails removed: 19,117
```

3.3 Create the 20,000-Email Project Sample

After basic data cleaning, a random sample of **20,000 emails** was selected from the remaining 497,679 emails. A fixed random seed (42) was used so that the same sample can be reproduced if needed.

Figure 7. Creation of the 20,000-email project sample.



```
1 # Create a reproducible random sample of 20,000 emails
2
3 project_sample = clean_pool.shuffle(seed=42).select(range(20000))
4
5 print(f'Clean pool: {len(clean_pool):,} emails')
6 print(f'Project sample: {len(project_sample):,} emails')
7
8 Clean pool: 497,679 emails
9 Project sample: 20,000 emails
```

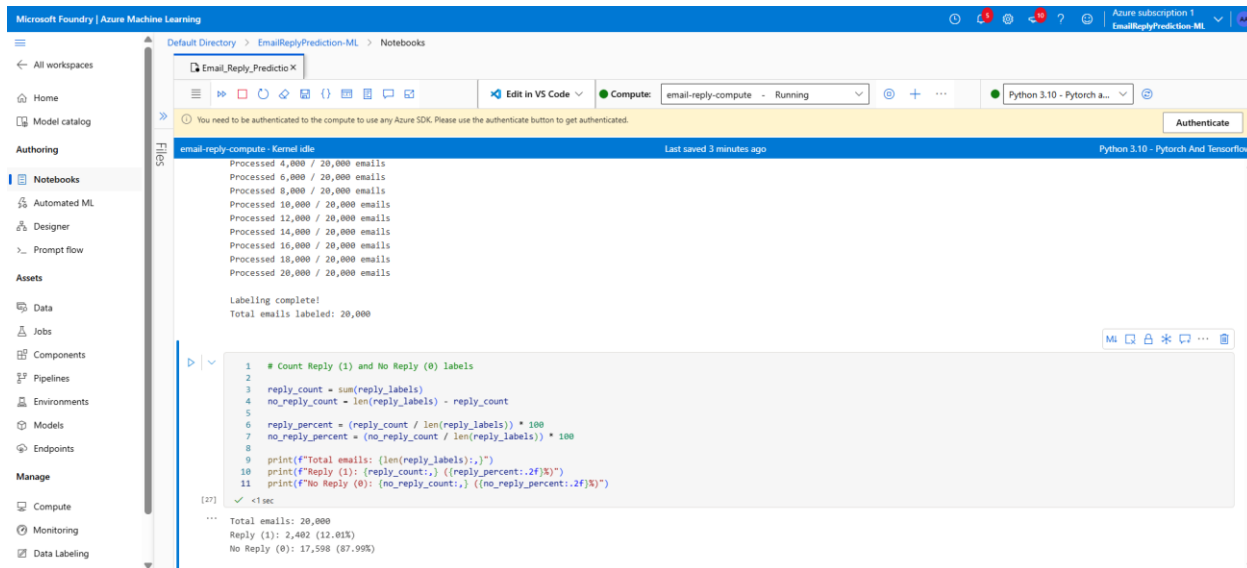
Step 4 — Email Reply Detection and Labeling

4.1 Create Reply and No-Reply Labels

The **20,000 sampled emails** were checked against the larger email reference pool to identify whether each email received a reply. A reply was identified using the **normalized subject, sender and recipient, email date, and a 30-day reply window**.

Emails with a matching reply were labeled **1 (Reply)**, while emails without a matching reply were labeled **0 (No Reply)**. The final result contained **2,402 Reply emails (12.01%)** and **17,598 No Reply emails (87.99%)**.

Figure 8. Reply and No-Reply label distribution for the 20,000-email sample.



```
email-reply-compute - Kernel idle
Processed 4,000 / 20,000 emails
Processed 8,000 / 20,000 emails
Processed 12,000 / 20,000 emails
Processed 16,000 / 20,000 emails
Processed 20,000 / 20,000 emails
Labeling complete!
Total emails labeled: 20,000

1 # Count Reply (1) and No Reply (0) labels
2
3 reply_count = sum(reply_labels)
4 no_reply_count = len(reply_labels) - reply_count
5
6 reply_percent = (reply_count / len(reply_labels)) * 100
7 no_reply_percent = (no_reply_count / len(reply_labels)) * 100
8
9 print(f"Total emails: {len(reply_labels)}")
10 print(f"Reply (1): {reply_count:,} ({reply_percent:.2f}%")
11 print(f"No Reply (0): {no_reply_count:,} ({no_reply_percent:.2f}%")

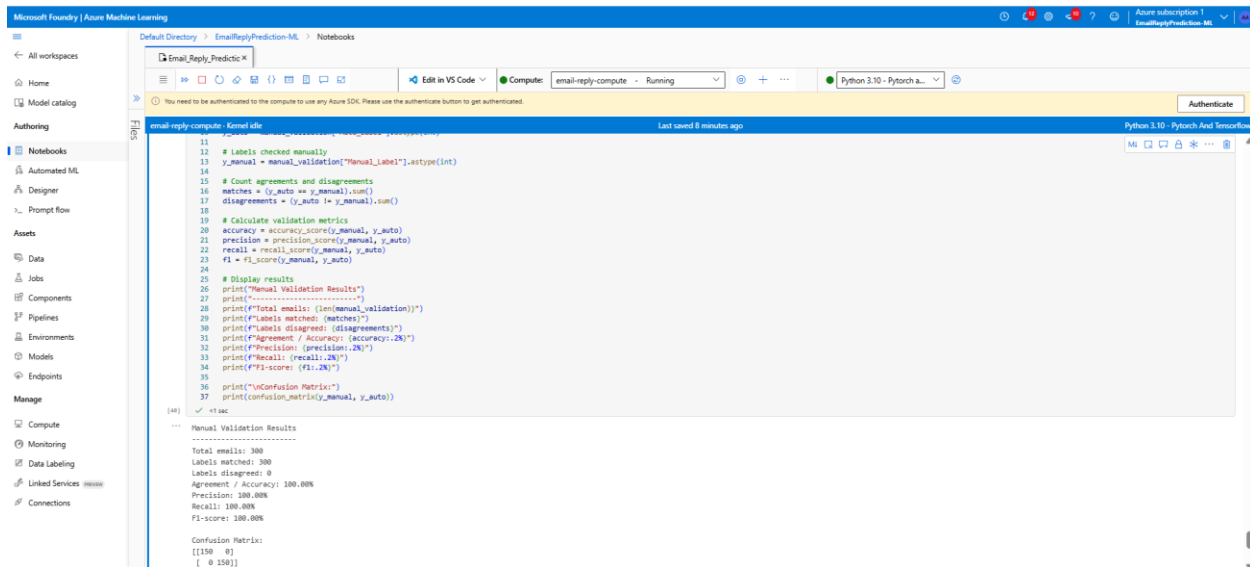
[27] ✓ <1 sec
...
Total emails: 20,000
Reply (1): 2,402 (12.01%)
No Reply (0): 17,598 (87.99%)
```

Step 4 — Email Reply Detection and Labeling

4.2 Manual Validation Results

A balanced sample of 300 emails was manually validated, including 150 Reply and 150 No Reply emails. All 300 manual labels matched the automatic labels, giving 100% agreement in the validation sample.

Figure 9. Manual validation results comparing automatic and manually reviewed email labels.



The screenshot shows the Microsoft Foundry interface with a notebook titled 'Email_Reply_Prediction'. The code in the notebook performs manual validation by comparing automatically predicted labels with manually reviewed labels. The output of the code is displayed below the code editor, showing the following results:

```
Manual Validation Results
-----
Total emails: 300
Labels matched: 300
Labels disagreed: 0
Agreement / Accuracy: 100.00%
Precision: 100.00%
Recall: 100.00%
F1-score: 100.00%

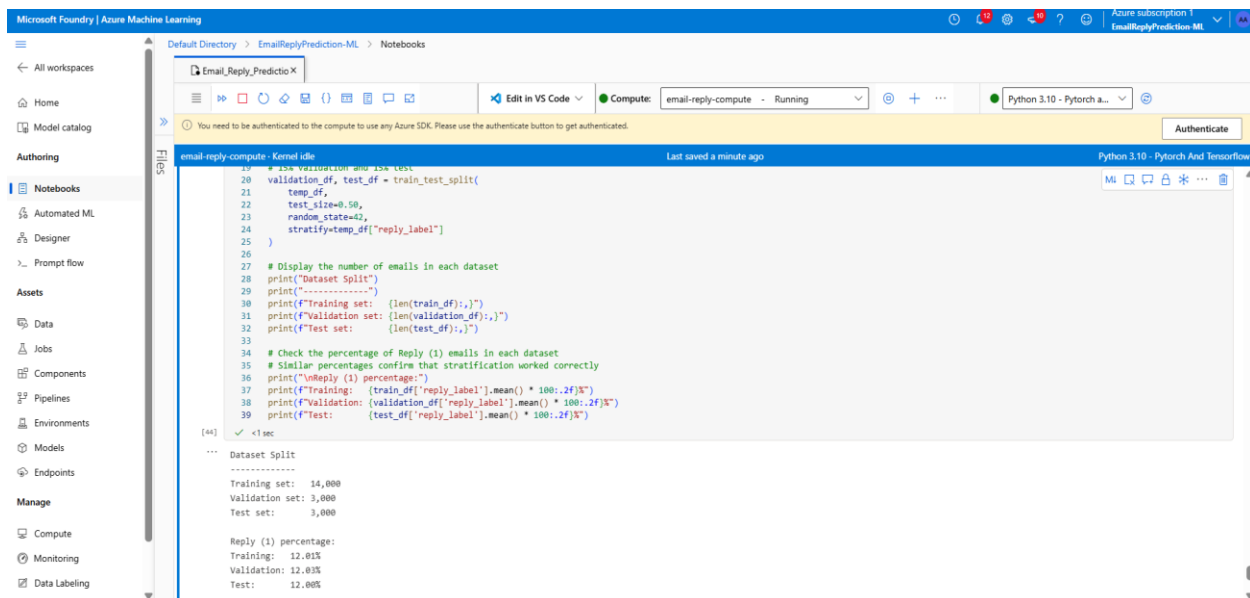
Confusion Matrix:
[[300  0]
 [  0 300]]
```

Step 5 — Data Preprocessing and Model Preparation

5.1 Model Input and Dataset Split

The email subject and body were combined to create the text input for model training. The 20,000 labeled emails were then divided using a stratified 70/15/15 split: 14,000 for training, 3,000 for validation, and 3,000 for testing. The Reply rate remained approximately 12% in all three sets.

Figure 10. Stratified training, validation, and test split of the 20,000-email dataset.



The screenshot shows the Microsoft Foundry interface with a notebook titled 'Email_Reply_Prediction'. The code in the notebook performs a stratified 70/15/15 split of the 20,000-email dataset. The output of the code is displayed below the code editor, showing the following results:

```
Dataset Split
-----
Training set: 14,000
Validation set: 3,000
Test set: 3,000

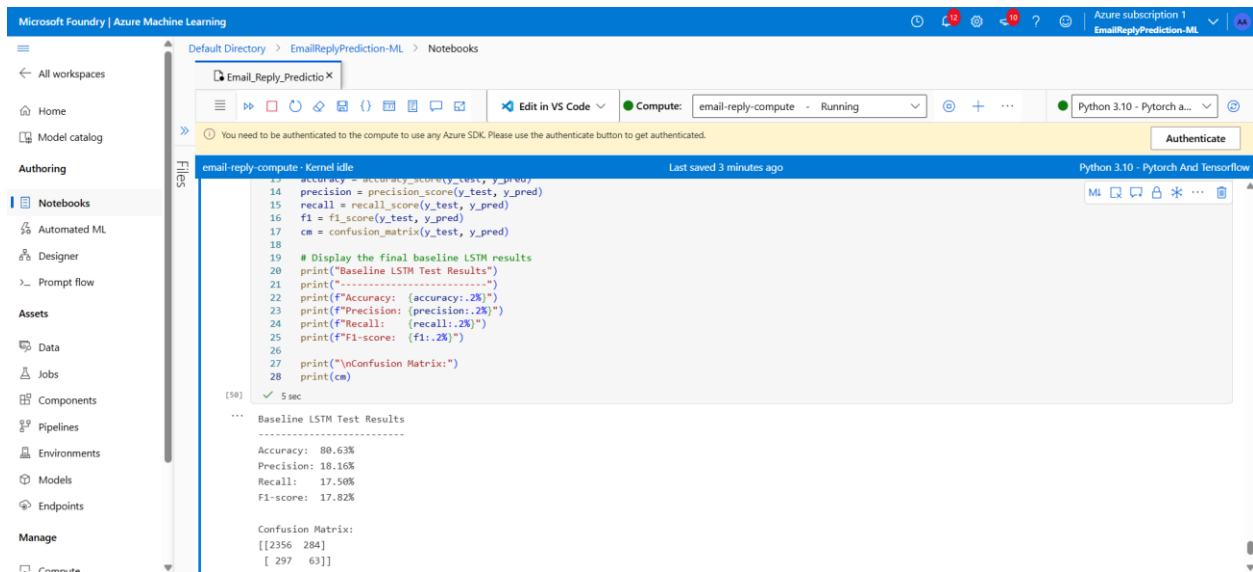
Reply (1) percentage:
Training: 12.01%
Validation: 12.03%
Test: 12.00%
```

Step 6 — LSTM Model

6.1 Baseline LSTM Evaluation

The baseline LSTM was evaluated on the 3,000-email test set. It achieved 80.63% accuracy, but Reply-class performance was low, with 18.16% precision, 17.50% recall, and 17.82% F1-score. This showed that accuracy alone was not sufficient for evaluating the imbalanced dataset and that further model improvement was required.

Figure 11. Baseline LSTM test performance and confusion matrix.

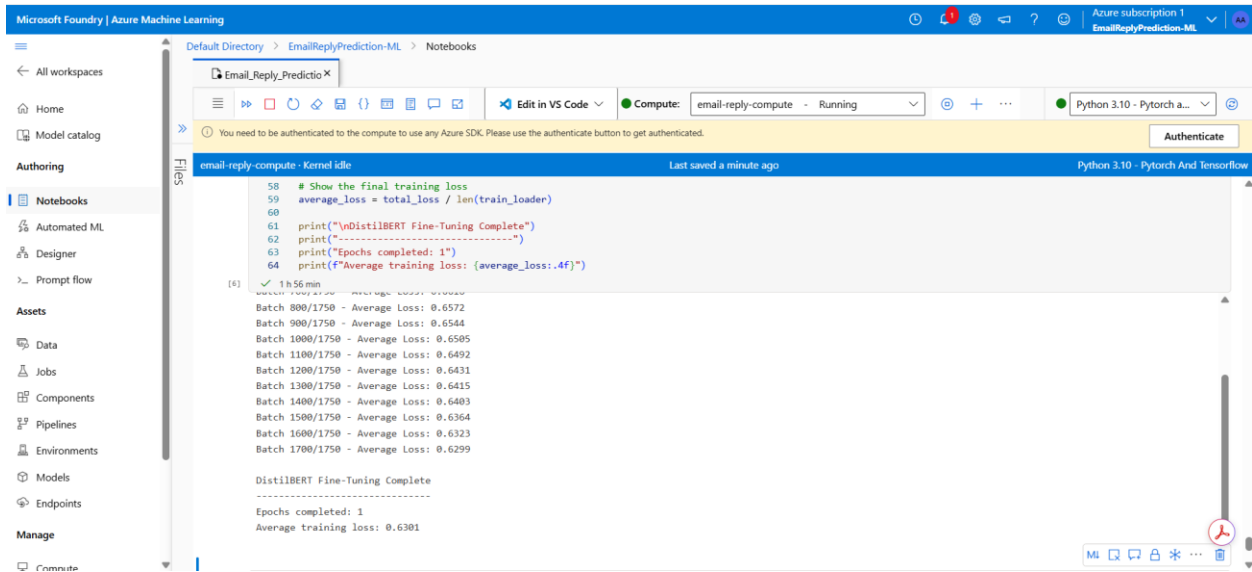


Step 7 — DistilBERT Model Development and Evaluation

Step 7.1 — DistilBERT Fine-Tuning

The pretrained DistilBERT model was fine-tuned using the **14,000 training emails** for one epoch. A **batch size of 8** and class weights were used to account for the imbalanced Reply and No Reply classes. Training completed with an **average loss of 0.6301**. The fine-tuned model and tokenizer were then saved for later evaluation and deployment.

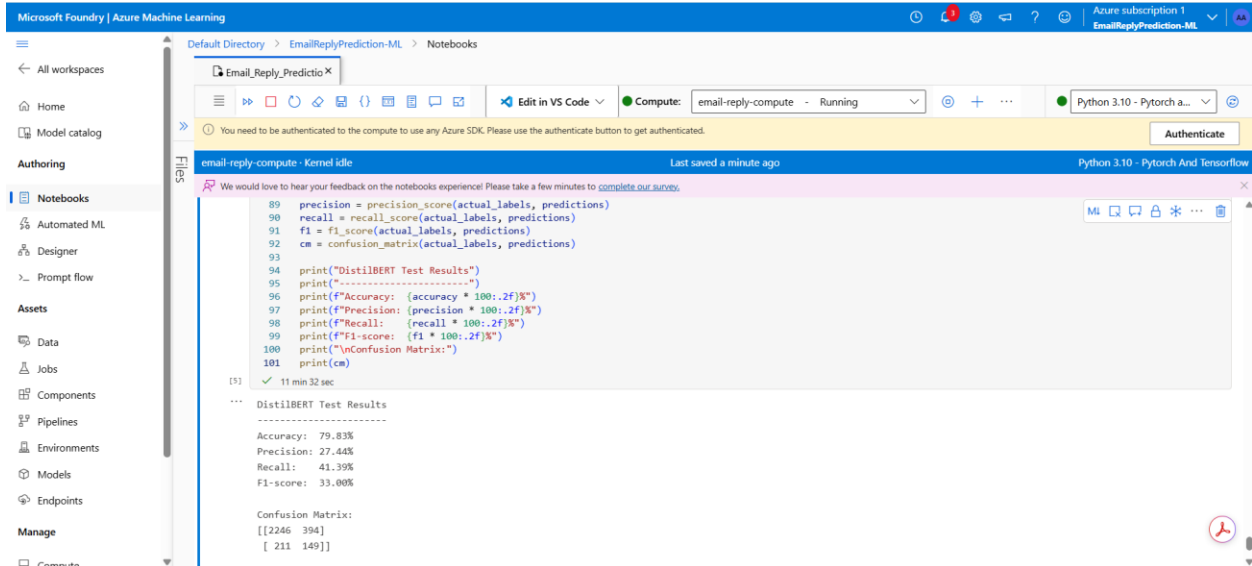
Figure 12. DistilBERT fine-tuning progress and final training loss.



Step 7.2 — DistilBERT Evaluation

The fine-tuned DistilBERT model was evaluated using the 3,000 unseen test emails. It achieved 79.83% accuracy, 27.44% precision, 41.39% recall, and 33.00% F1-score. The model correctly identified 149 Reply emails.

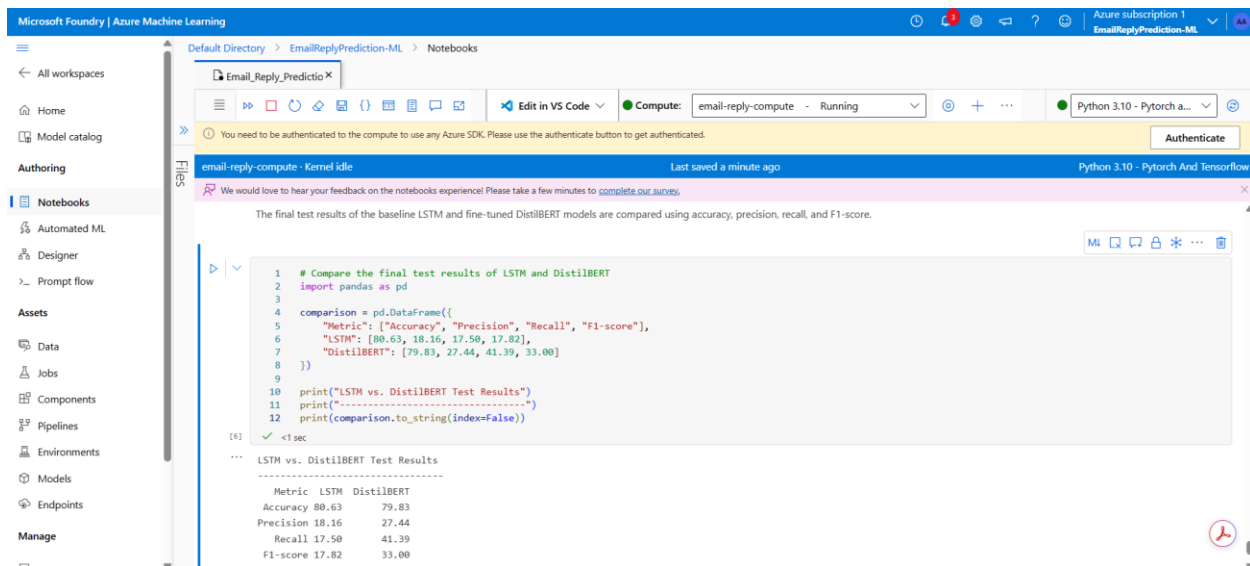
Figure 13. DistilBERT test-set evaluation results.



Step 8 — LSTM vs. DistilBERT Model Comparison

The final test results of the LSTM and DistilBERT models were compared. The LSTM achieved slightly higher accuracy, while DistilBERT achieved higher precision, recall, and F1-score. DistilBERT correctly detected more Reply emails, which is important because the dataset contains fewer Reply emails than No Reply emails.

Figure 14. LSTM and DistilBERT test-result comparison.

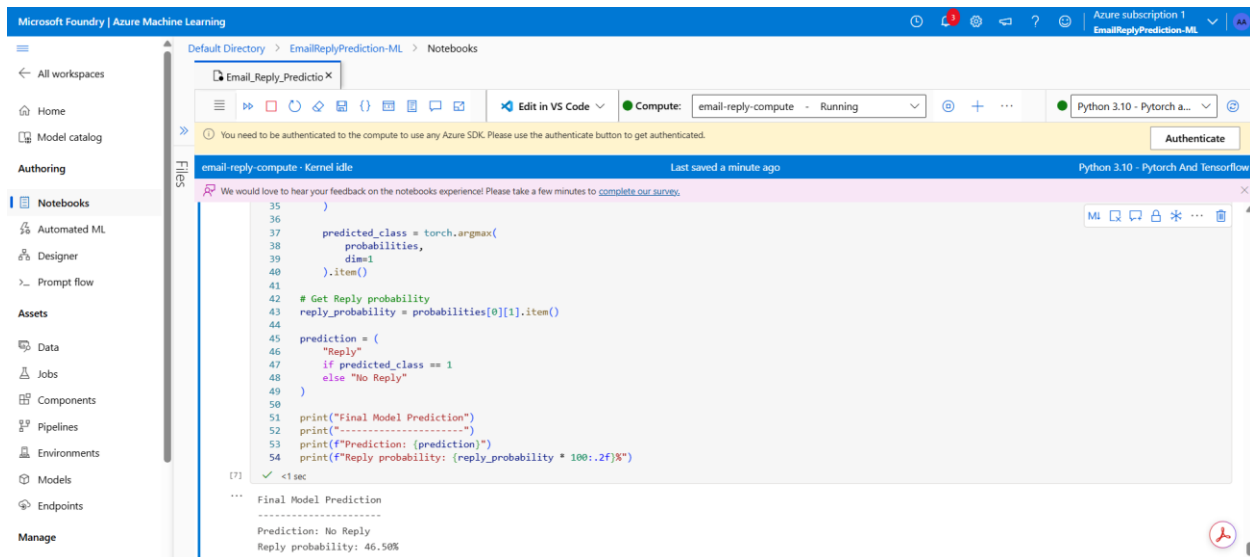


Step 9 — Final Model Preparation

9.1 — Final Model Prediction

The fine-tuned DistilBERT model was tested using a new sample email before deployment. The model predicted **No Reply**, with a **46.50% Reply probability**, confirming that the saved model can process new email text and generate a prediction.

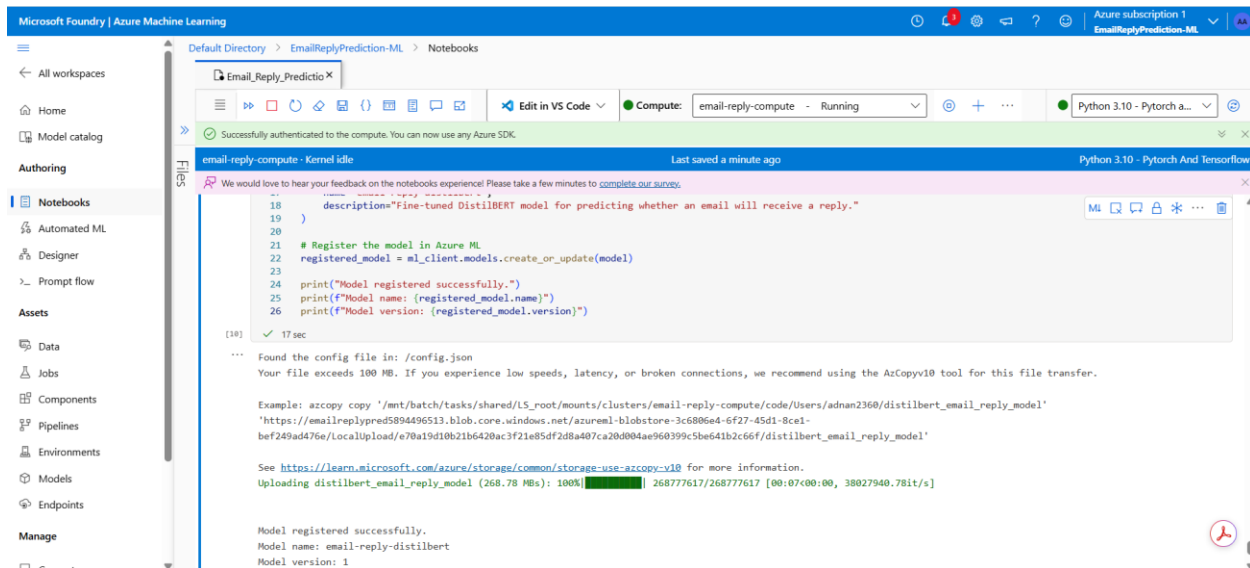
Figure 15. Final DistilBERT prediction on a new email.



9.2 — Azure ML Model Registration

The fine-tuned DistilBERT model was registered in Azure Machine Learning as a custom model. The registered model was named **email-reply-distilbert** and saved as **Version 1**, making it available for deployment.

Figure 16. Fine-tuned DistilBERT model successfully registered in Azure Machine Learning.



Step 10 — Prototype Model Deployment

The fine-tuned DistilBERT model was successfully deployed as a **prototype managed online endpoint** in Azure Machine Learning. The endpoint allows new email text to be sent to the deployed model for real-time prediction. The deployed model was tested using a **new email that was not part of the training dataset** and predicted **No Reply** with a **41.84% Reply probability**.

Figure 17. Fine-tuned DistilBERT model successfully deployed and tested using an Azure Machine Learning managed online endpoint.

